Principles of Data Science

Ex8: Interactive Dashboards with Tableau

Learning Objective:

To understand and implement interactive data visualization and dashboard design using Tableau with the Sample Superstore dataset. The objective is to create meaningful visualizations such as bar charts, line charts, and scatter plots to analyze sales, profit, and regional performance. It also aims to develop skills in applying filters, dashboard actions, and interactive features to explore data effectively and support data-driven decision-making.

Built-in Functions:

SUM() – Calculates the total sales and profit.

YEAR() – Extracts the year from the order date.

MONTH() – Extracts the month from the order date.

AVG() – Calculates the average sales or profit.

COUNT() – Counts the number of records.

IF() – Applies conditional logic.

FILTER() – Filters data based on specified conditions.

SORT() – Arranges data in ascending or descending order. MAX() – Returns the maximum value of sales or profit.

MIN() – Returns the minimum value of sales or profit.

COUNTD() – Counts distinct values in a field.

ROUND() – Rounds a number to a specified number of decimal places.

DATEDIFF() – Calculates the difference between two dates.

DATEPART() – Extracts a specific part of a date.

ATTR() – Returns a single value for a field when possible.

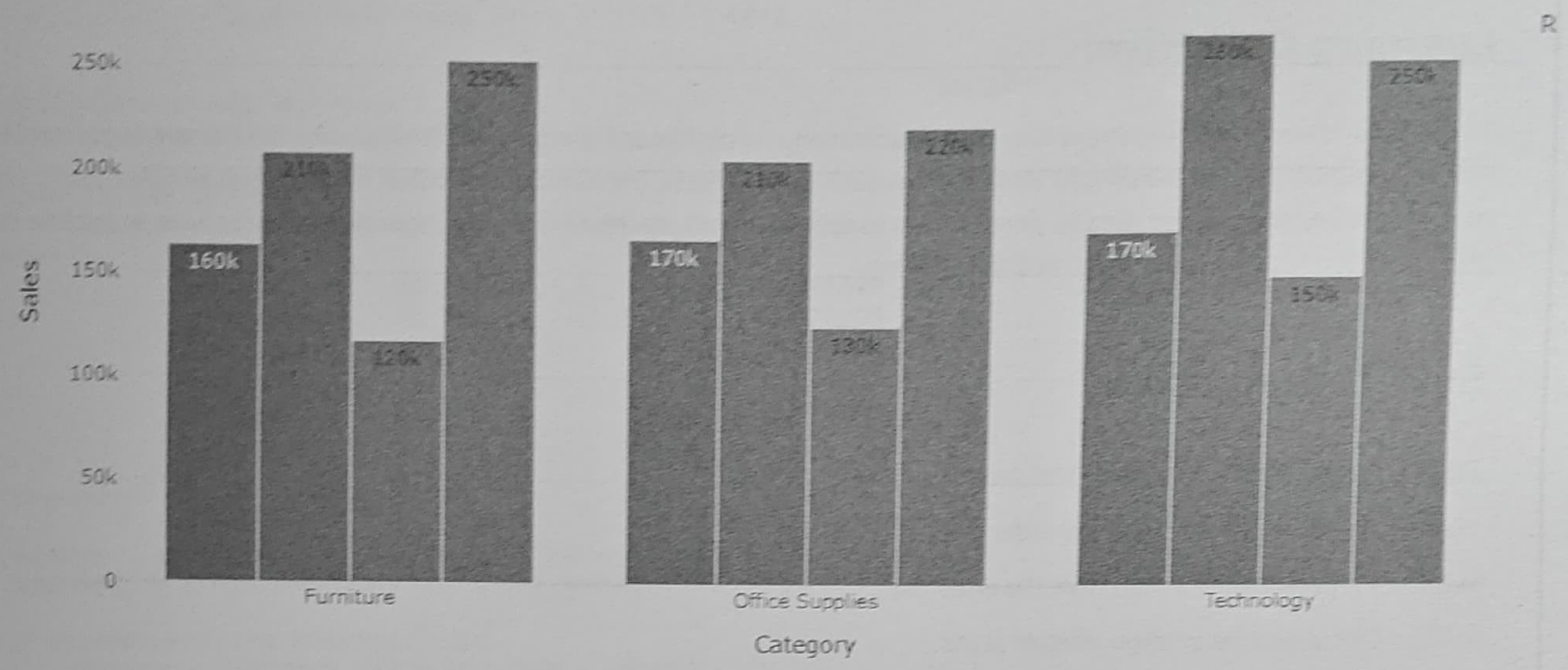
ZN() – Replaces null values with zero.

```
import pandas as pd
import plotly.express as px
import plotly.graph_objects as go
from dash import Dash, dcc, html, Input, Output
df = pd.read_csv("/content/SampleSuperstore.csv", encoding="latin1")
df.columns = [c.replace('_', ' ').replace('-', ' ').title() for c in df.columns]
date_cols = [c for c in df.columns if "Date" in c]
if date_cols:
    target_col = date_cols[0]
    df[target_col] = pd.to_datetime(df[target_col])
    df["Year-Month"] = df[target_col].dt.to_period("M").astype(str)
else:
    df["Year-Month"] = "Unknown"
print("Dataset successfully loaded! Current columns:")
print(list(df.columns))
```

```
Dataset successfully loaded! Current columns:
['Ship Mode', 'Segment', 'Country', 'City', 'State', 'Postal Code', 'Region', 'Category', 'Sub Category', 'Sales', 'Quantity
```

```
bar_data = df.groupby(
    ["Category", "Region"], as_index=False
)["Sales"].sum()
fig = px.bar(
    bar_data,
    x="Category",
    y="Sales",
    color="Region",
    barmode="group",
    text_auto=".2s",
    title="Sales by Category"
)
```

```
fig.update_layout(template="plotly_white")
fig.show()
```



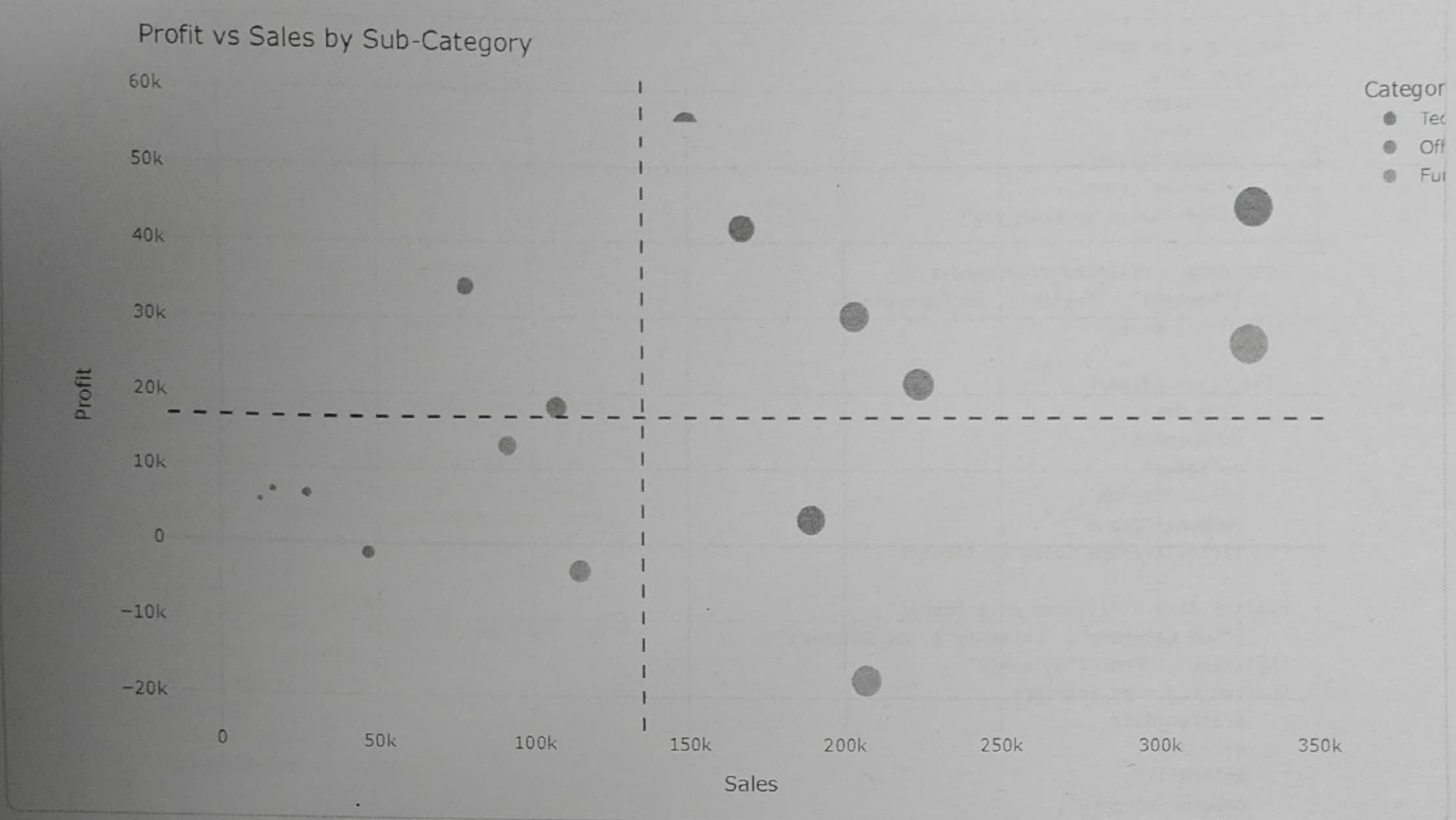

```
trend_data = df.groupby(['Segment', 'Region'], as_index=False)[['Sales', 'Profit']].mean()
fig = px.bar(
    trend_data,
    x="Segment",
    y="Profit",
    color="Region",
    barmode="group",
    title="Average Profit by Segment & Region"
)

fig.update_layout(
    template="plotly_white",
    xaxis_title="Segment",
    yaxis_title="Average Profit"
)
fig.show()
```




```
dfg = df.groupby(['Sub Category', 'Category'], as_index=False)[['Sales', 'Profit']].sum()
fig = px.scatter(dfg, x='Sales', y='Profit', color='Category', size='Sales', hover_name='Sub Category', title='Profit vs S
fig.add_hline(y=dfg['Profit'].mean(), line_dash='dash')
fig.add_vline(x=dfg['Sales'].mean(), line_dash='dash')
```

```
fig.update_layout(template='plotly_white', margin=dict(l=10, r=10, t=30, b=10))
fig.show()
```




```
app = Dash(__name__)
regions = sorted(df["Region"].dropna().unique())
categories = sorted(df["Category"].dropna().unique())
segments = sorted(df["Segment"].dropna().unique())
app.layout = html.Div([
    html.H1(
        "Sales and Profit Dashboard",
        style={"textAlign": "center", "fontFamily": "sans-serif"}
    ),
    html.Label("Select Region"),
    dcc.Dropdown(
        id="region-filter",
        options=[{"label": r, "value": r} for r in regions],
        value=regions,
        multi=True
    ),
    html.Label("Select Category"),
    dcc.Dropdown(
        id="category-filter",
        options=[{"label": c, "value": c} for c in categories],
        value=categories,
        multi=True
    ),
    html.Label("Select Segment"),
    dcc.Dropdown(
        id="segment-filter",
        options=[{"label": s, "value": s} for s in segments],
        value=segments,
        multi=True
    ),
    dcc.Graph(id="bar-chart"),
    dcc.Graph(id="line-chart"),
    dcc.Graph(id="scatter-chart")
])
@app.callback(
    Output("bar-chart", "figure"),
    Output("line-chart", "figure"),
    Output("scatter-chart", "figure"),
    Input("region-filter", "value"),
    Input("category-filter", "value"),
    Input("segment-filter", "value")
)
def update_dashboard(selected_regions, selected_categories, selected_segments):
    filtered_df = df[
        df["Region"].isin(selected_regions or []) &
        df["Category"].isin(selected_categories or []) &
        df["Segment"].isin(selected_segments or [])
    ].copy()
```

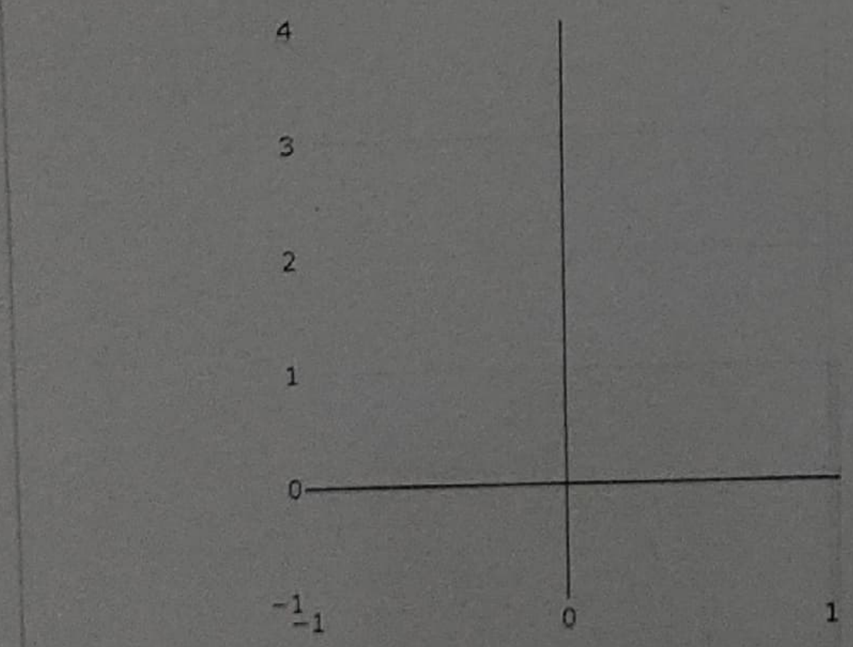
```
    ["Category", "Region"], as_index=False
)["Sales"].sum()

bar_fig = px.bar(
    bar_data,
    x="Category",
    y="Sales",
    color="Region",
    barmode="group",
    title="Sales by Category"
)
line_data = filtered_df.groupby(
    ["Segment", "Region"], as_index=False
)["Sales"].mean()

line_fig = px.bar(
    line_data,
    x="Segment",
    y="Sales",
    color="Region",
    barmode="group",
    title="Average Sales by Segment"
)
scatter_data = filtered_df.groupby(
    ["Sub Category", "Category"], as_index=False
)[["Sales", "Profit"]].sum()
scatter_fig = px.scatter(
    scatter_data,
    x="Sales",
    y="Profit",
    color="Category",
    size="Sales",
    hover_name="Sub Category",
    title="Profit vs Sales by Sub-Category"
)
    for fig in [bar_fig, line_fig, scatter_fig]:
        fig.update_layout(template="plotly_white")
    return bar_fig, line_fig, scatter_fig
if __name__ == "__main__":
    print("Starting Dash server...")
    app.run(debug=True, port=8050)
```

```
Starting Dash server...
```

Sales and Profit Dashboard



Errors

ID not found in layout	8:25:53 AM
ID not found in layout	8:25:53 AM
ID not found in layout	8:25:53 AM

Plotly Cloud | Errors 3 | Callbacks | v4.4.1 | Server

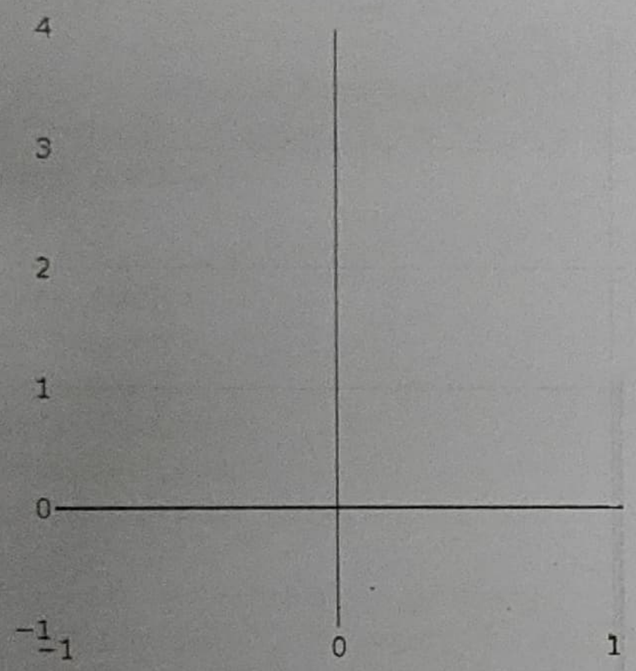
```
app = Dash(__name__)
```

```
    html.Div([
        dcc.Graph(id="bar-chart"),
        dcc.Graph(id="line-chart"),
        dcc.Graph(id="scatter-chart")
    ])
])
```

```
@app.callback(
    Output("bar-chart", "figure"),
    Output("line-chart", "figure"),
    Output("scatter-chart", "figure"),
    Input("region-filter", "value"),
    Input("category-filter", "value"),
    Input("year-filter", "value")
)
def update_dashboard(regions, categories, years):
    filtered_df = df[
        df["Region"].isin(regions or []) &
        df["Category"].isin(categories or []) &
        df["Order Date"].dt.year.isin(years or [])
    ]
    bar_fig = px.bar(
        filtered_df.groupby(
            ["Category", "Region"], as_index=False
        )["Sales"].sum(),
        x="Category",
        y="Sales",
        color="Region",
        title="Sales by Category"
    )
    line_fig = px.line(
        filtered_df.groupby(
            ["Year-Month", "Region"], as_index=False
        )["Sales"].sum(),
        x="Year-Month",
        y="Sales",
        color="Region",
        title="Sales Trend Over Time"
    )
    scatter_fig = px.scatter(
        filtered_df.groupby(
            ["Sub-Category", "Category"], as_index=False
        )[["Sales", "Profit"]].sum(),
        x="Sales",
        y="Profit",
        color="Category",
        hover_name="Sub-Category",
        title="Profit vs Sales by Sub-Category"
    )
    return bar_fig, line_fig, scatter_fig
```

```
if __name__ == "__main__":
    app.run(debug=True, port=8050)
```

Sales and Profit Dashboard



Errors

ID not found in layout 8:27:42 AM

ID not found in layout 8:27:42 AM

ID not found in layout 8:27:42 AM

Plotly Cloud | Errors 3 | Callbacks | v4.4.1 | Server

Learning Outcome:

1.Understand how to load and explore datasets in Tableau.

2.Create bar charts, line charts, and scatter plots.

3.Design interactive dashboards using multiple worksheets.

4.Apply filters and dashboard actions for dynamic data exploration.

5.Analyze sales trends, regional performance, and profitability.